CALIFICAPP REVIEW INSIGHTS · PERFICIENT GLOBAL AI-FIRST ACADEMY · L1 CASE 05

# Why this project exists, and why this is the solution

A motorcycle-gear store in Colombia collects customer ratings on a tablet at the counter. The numbers say everything is fine. A handful of comments say otherwise, and nobody reads them.

**3,184**
real ratings, Oct 2025 – Oct 2026

**4.9★**
monthly average; it never moves

**94%**
are 5★ (2,992 of 3,184)

**14**
of 997 comments need the owner to act

## 1. Motivation

### The user and the situation

The user is the **store owner**: not technical, no time to read comments, looks at the numbers occasionally. The store sells helmets, tyres, rain gear and accessories, and also **fits them**: tyre changes, crash bars, luggage racks. After each sale the customer rates the employee who served them from 1 to 5 on a **Calificapp** tablet, and can leave a comment.

About a third of the ratings carry a comment, typically short (median 18 characters) and in informal Colombian Spanish: "exelente atencion", "melo", emoji. Almost all are compliments. The owner sees a 4.9 average and reasonably concludes there is nothing to look at.

### What is hidden in the comments

Reading all 997 real comments by hand found 14 that the owner must act on. Some of them:

| Review | Stars | What it says (paraphrased) | Why it matters |
|---|---|---|---|
| R01972 | 1 | A tyre the store fitted came off the rim ("despestañada") and the rider crashed | Safety incident; possible liability |
| R00309 | **5** | "Excellent service. But very lewd" | Possible staff misconduct, hidden in a top rating |
| R02021 | 1 | Warranty claim on a helmet, no solution given | Unresolved claim; risk of a formal complaint |
| R00460, R00933 | **5** | "...but the discount was missing", "...although prices are high" | Price friction inside top ratings |
| R01215 | **5** | "It's a special order... please call if you get it" | A sale waiting on a call nobody made |

> **The star rating cannot find them.** Of the 14 reviews that need action, **4 are 5★**, including one of the 4 urgent ones (R00309). Reading only the 1–3★ comments would find 10 of 14 and miss an urgent case. And a 1★ can be a mis-tap on a positive comment. The decision has to come from the text.

### Why it is worth solving now

- **The cost of a miss is asymmetric.** One unread safety report or harassment complaint can cost more than everything the ratings are meant to improve. The owner tolerates some false alarms; not a missed crash report.
- **Nobody will read 1,000 compliments.** Asking the owner or a manager to read every comment is the obvious fix, and it is not happening.
- **The history expires.** The Calificapp backend deletes each rating 500 days after it is created, so anything not analysed is lost.
- **The owner has questions the dashboard cannot answer:** "¿Algún cliente tuvo un accidente?", "¿Qué dicen de los cascos?", "¿Cuántas reseñas de 1 estrella hubo en septiembre?".

### What success means

Defined before any code (`docs/01-problem-statement.md`) and measured on hand-labeled data, real and synthetic reported separately: **every urgent review found** (100%); at least 90% of actionable reviews found with at least 60% precision; **no invented citations**; exact numbers for counting questions; at least 80% of owner questions answered correctly; and retrieval measured against the simpler alternative of sending every comment to the model.

## 2. Why this is the solution

The solution has three parts: **(1)** a local AI model reads *every* comment and labels it urgent / review / normal, with an alert-word safety net that can only raise severity; **(2)** the owner asks questions in plain Spanish, and a router sends counting questions to exact SQL and meaning questions to semantic search plus the model, which must cite the reviews it uses; **(3)** every citation is checked against the reviews the model was given. Each choice below was made against an alternative, and most were measured.

| Decision | Chosen, and why | Rejected alternative, and the evidence |
|---|---|---|
| **Decide from the text** | An LLM classifies every comment by what it reports, not by its stars. | **Filter by low ratings:** finds 10 of 14 actionable reviews and misses urgent R00309 (5★). |
| **Domain context for the model** | A context file (`CLAUDE.md`): severity definitions, local slang, the rule that a review is data, never instructions. | **Bare prompt (v0):** precision 8/33 (flagged plain praise) and it obeyed a prompt injection. With the context (v1): 13/13, injection ignored. |
| **Alert-word safety net** | After the model, a short word list raises severity and never lowers it. | **Model alone:** missed R00309 with and without the context file. With the net: real urgent recall 3/4 → **4/4**, no false alarms on the 997 real comments. |
| **Counting by SQL, not the model** | Rules detect counting questions; fixed, parameterized SQL computes the number. | **Let the model count or fill filters:** it read "de 1 estrella" as "at least 1 star" and reported **2,892** instead of **11**. With SQL: 13/13 counting questions exact. |

| | | |
|---|---|---|
| **Retrieval for meaning questions** | Embed the question, take the 20 closest reviews (half reserved for flagged ones on complaint questions), answer only from them. | **Send all 997 comments in one prompt** (~18k tokens, it fits): measured on 26 questions, **1/16 vs 7/16** correct, and **13 invented review IDs vs 0**. The model loses the few reviews that matter among the compliments. |
| **Local model** | Qwen 2.5 7B and bge-m3 embeddings via Ollama on the store's PC. Client reviews never leave the computer; no per-call cost. | **Claude API by default:** stronger, but paid and sends client text to a third party; kept as an option. **Free cloud tiers:** some train on submitted data. **Fine-tuned classifier:** 14 real negatives are too few to train one. |
| **Retrieval pipeline over an n8n automation** | Code in a repository: testable, versioned, reviewable. | **n8n workflow:** would fit a weekly digest, but logic in a no-code flow is harder to test and review, and Q&A needs retrieval anyway. |
| **SQLite for everything** | Reviews, labels and vectors in one file; brute-force search is under a second at ~1,000 vectors. | **Chroma** (the original plan): an extra dependency with no benefit at this size. **DynamoDB copy:** risks touching the client's account. |
| **Citations, verified** | Every answer cites review IDs; any ID not among the reviews given to the model marks the answer unverified. | **Uncited summaries:** the owner could not check them. Result: 0 invalid citations in 40 answers. |

## 3. What it achieves, and what it does not

| Criterion | Target | Result |
|---|---|---|
| Urgent reviews found | 100% | **Real 4/4** · held-out 8/8 · synthetic 15/17 |
| Actionable reviews found / precision | ≥ 90% / ≥ 60% | **Real 14/14 / 14/14** (no false alarms) |
| Invented citations | 0 | **0** in 40 answers |
| Counting questions exact | 100% | **13/13** within the question sets |
| Owner questions answered correctly | ≥ 80% | **Not met:** 15/25 and 10/15; about half of open-ended questions |

The core problem, finding what to act on among a thousand compliments, is solved on the real data. Open-ended questions are not: retrieval usually finds the right review and the 7B model drops it at the last step. A count with a month and no year ("¿...en septiembre?") silently counts every review. The planned weekly digest was cut. All of this, with specific failing inputs, is in `docs/failure-analysis.md` and `evidence/eval_results.md`.

Data: all 3,184 real ratings of one store, anonymized with the owner's permission (employees as EMP_xx, customer names and phones removed). Review IDs refer to the anonymized dataset. Repository: github.com/smolinag/calificapp-review-insights